

第 1 集 · 约 15 分钟

cargo 和 workspace: 先把项目搭起来

一份加密代码, 怎么同时编进 exe 和浏览器

配套阅读: 书的第 1 章



这一集你会学到

1. workspace、crate、module 三层结构, 和 Python 的包怎么对应
2. Cargo.toml 和 Cargo.lock: Rust 版的 requirements.txt 和 pip freeze
3. edition 是什么, 为什么 2024 要你多写几个 unsafe
4. 分清两种报错: 编译器报的, 和链接器报的

一个问题

手机用 `http://192.168.x.x` 打开页面，浏览器连加密 API 都不给。端到端加密还怎么做？

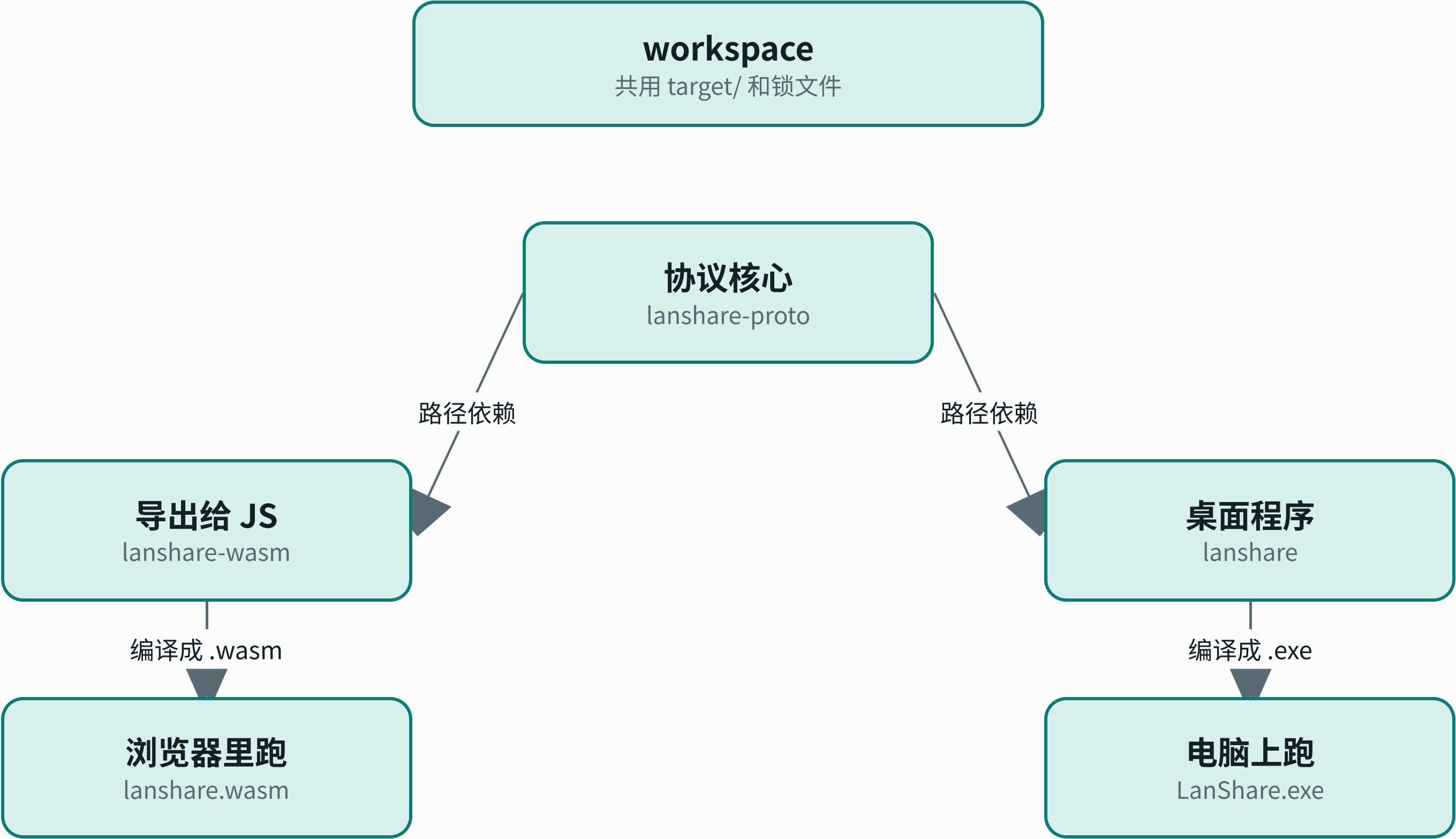
自己带一份加密代码进浏览器：同一份 Rust，既编进 `LanShare.exe`，也编成 `lanshare.wasm`。

v2 要做到的三件事

这三件事，Python 都不太好办

- 浏览器里也要能加密
Rust 能编译成 WebAssembly（WASM），在浏览器里以接近原生的速度运行
- 服务端和浏览器用同一份加密代码
同一个函数既编进 exe 又编进 wasm，两端行为一致，测试一次两端受益
- 小、快、单文件
编译出来就是机器码，没有解释器，也不用“先解压再启动”

一个仓库三个 crate



根目录的配置

Cargo.toml

```
1 [workspace]
2 resolver = "3"
3 members = ["crates/lanshare-proto", "crates/lanshare-wasm", "crates/lanshare"]
4
5 [workspace.package]
6 version = "2.0.0"
7 edition = "2024"
8 publish = false
9
10 [profile.release]
11 opt-level = 3
12 lto = true
13 codegen-units = 1
14 strip = true
15 # ...
```

成员 crate 的配置

crates/lanshare-wasm/Cargo.toml

```
1 [package]
2 name = "lanshare-wasm"
3 description = "把 lanshare-proto 导出给浏览器 (裸 C ABI, 不依赖 wasm-bindgen) "
4 version.workspace = true
5 edition.workspace = true
6 publish.workspace = true
7
8 [lib]
9 crate-type = ["cdylib", "rlib"]
10
11 [dependencies]
12 lanshare-proto = { path = "../lanshare-proto" }
13 rand_core = { version = "0.6", features = ["getrandom"] }
14
15 [target.'cfg(target_arch = "wasm32")'.dependencies]
16 getrandom = { version = "0.2", features = ["custom"] }
```

pip vs cargo

Python

```
1 python -m venv .venv
2 pip install requests
3 pip freeze > requirements.txt
4 python main.py
```

Rust

```
1 # 不需要虚拟环境
2 cargo add request
3 # Cargo.lock 自动生成、自动更新
4 cargo run
```

探针撞上的四个报错

● ● ● Windows PowerShell

```
PS D:\lanshare> cargo build -p lanshare-wasm --target wasm32-unknown-unknown
```

```
error: extern blocks must be unsafe
```

```
error: unsafe attribute used without unsafe
```

```
error: unsafe attribute used without unsafe
```

```
PS D:\lanshare> cargo build -p lanshare-wasm --target wasm32-unknown-unknown
```

```
rust-ldd: error: ...wprobe...rcgu.o: undefined symbol: ls_random
```

改之前，改之后

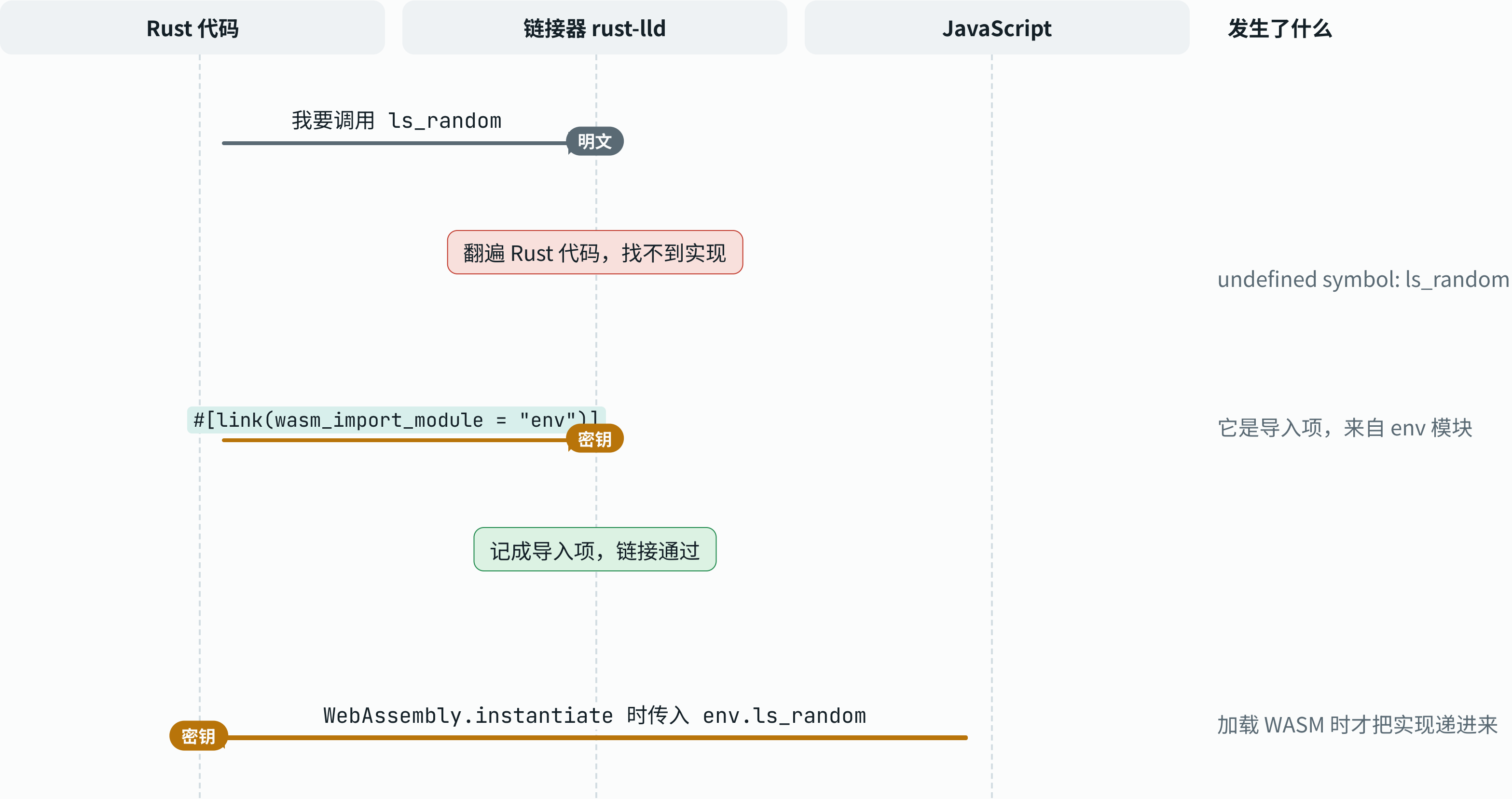
改之前 (2021 的习惯)

```
1 extern "C" { fn ls_random(ptr: *mut u8, len: usize); }
2
3 #[no_mangle]
4 pub extern "C" fn probe_seal(len: usize) -> usize { /* ..
```

改之后 (2024)

```
1 #[cfg(target_arch = "wasm32")]           // 只在编译到
2 #[link(wasm_import_module = "env")]       // 这个函数由宿
3 unsafe extern "C" { fn ls_random(ptr: *mut u8, len: usize);
4
5 #[unsafe(no_mangle)]
6 pub extern "C" fn probe_seal(len: usize) -> usize { /* ..
```

ls_random 在哪



动手：条件编译

目标平台：

在 Windows 上编译

在 Linux 上编译

Rust：编译时选择

```
1  #[cfg(windows)]
2  pub fn shell_open(target: &str) { /* 调 ShellExecuteW */ }
3
4  #[cfg(not(windows))]
5  pub fn shell_open(target: &str) { /* 调 xdg-open */ }
```

划掉的那段**不会被编进程序**（只要求语法正确，类型检查都跳过）：在 Windows 上编译出来的程序里只有一个 shell_open。

Python：运行时判断

```
1  if os.name == "nt":
2      os.startfile(target)
3  else:
4      subprocess.run(["xdg-open", target])
```

两段代码都在程序里，跑到这一行才判断走哪段。

选择题

一个 workspace 里的几个 crate，共用的是什么？

A 同一个 main.rs 入口

B 同一个 edition，必须统一

C 同一个 target/ 编译目录和同一个 Cargo.lock

D 同一个 Cargo.toml，成员不用再写自己的

选择题

一个 workspace 里的几个 crate，共用的是什么？

A 同一个 main.rs 入口

每个 crate 有自己的入口：库是 lib.rs，程序是 main.rs。

B 同一个 edition，必须统一

edition 是每个 crate 自己声明的，可以各不相同。本项目统一用 2024，是我们自己选的。

C 同一个 target/ 编译目录和同一个 Cargo.lock

对。所以全仓库的依赖版本一致，重复的依赖也只编译一次。

D 同一个 Cargo.toml，成员不用再写自己的

每个成员都有自己的 Cargo.toml，只是可以用 `version.workspace = true` 这种写法继承公共字段。

选择题

undefined symbol: `ls_random` 最准确的解释是哪个？

A 忘了写 `ls_random`，补一个 Rust 实现就好

B 没告诉链接器，它是从 `env` 模块导入的

C 2024 edition 要求它标成 `unsafe`

D 浏览器里的 WASM 根本用不了随机数

选择题

undefined symbol: `ls_random` 最准确的解释是哪个？

A 忘了写 `ls_random`，补一个 Rust 实现就好

它本来就不该由 Rust 实现。浏览器里的 WASM 拿不到系统随机源，得由 JavaScript 在加载时传进来。

B 没告诉链接器，它是从 `env` 模块导入的

对。加上 `#[link(wasm_import_module = "env")]`，链接器就把它记成一个从 `env` 导入的函数。

C 2024 edition 要求它标成 `unsafe`

那是前三个报错，编译阶段就报了。这一个开头是 `rust-lld`，是链接阶段的问题。

D 浏览器里的 WASM 根本用不了随机数

用得了，只是要靠 JavaScript 提供，JS 那边用的是 `crypto.getRandomValues`。

选择题

为什么 2024 edition 要求把下面这行写成 `unsafe extern "C"` ?

```
1 extern "C" { fn ls_random(ptr: *mut u8, len: usize); }
```

A 调用外部函数比较慢，要提醒一下

B unsafe 代码就是危险代码，写出来是为了警告别人别碰

C 外部函数的签名是程序员自己声明的，编译器没法验证它和真实实现一致

D 为了让函数名原样导出，不被改名

选择题

为什么 2024 edition 要求把下面这行写成 `unsafe extern "C"` ？

```
1 extern "C" { fn ls_random(ptr: *mut u8, len: usize); }
```

A 调用外部函数比较慢，要提醒一下

unsafe 和快慢没关系，它标的是“编译器没法替你检查”的地方。

B unsafe 代码就是危险代码，写出来是为了警告别人别碰

unsafe 的意思是“程序员担保这里正确”。跨语言边界就得用它，本项目把它限制在 WASM 边界和 Windows API 调用这几处。

C 外部函数的签名是程序员自己声明的，编译器没法验证它和真实实现一致

对。声明错了（比如参数类型不对）就是未定义行为。标成 unsafe 表示“我担保”，审查代码时一眼就能找到这些地方。

D 为了让函数名原样导出，不被改名

那是 no_mangle 的作用，对应的是第二、第三个报错。

想一想

有人说：“LanShare 从 Python 换成 Rust，所以更安全了。”这话对吗？v2 的安全提升到底从哪来？

想一想

有人说：“LanShare 从 Python 换成 Rust，所以更安全了。”这话对吗？v2 的安全提升到底从哪来？

不对。Python 本身也是内存安全的语言，换语言本身不会让它更安全。v2 真正的安全提升来自协议设计，也就是端到端加密。Rust 的作用是让这套设计能落地：同一份加密代码，既能编译到浏览器（WASM），又能编译到电脑，而且又小又快。

这一集我们学了

- ❶ workspace 管多个 crate，共用 target/ 和 Cargo.lock；crate 是编译的基本单位
- ❷ Cargo.toml 的依赖对应 requirements.txt，Cargo.lock 自动锁版本，也不用 venv
- ❸ cdylib 让 lanshare-wasm 编出浏览器能加载的 .wasm
- ❹ edition 由每个 crate 自己声明；2024 要求把编译器管不了的地方标成 unsafe
- ❺ 编译器报错和链接器报错是两回事，先看清是谁在喊

下一集

下一集：协议核心，借加密代码讲所有权、Result 和 trait